

CARA A - STATIC + STRINGS + STRING TREES

EXAM ANSWER TEMPLATE
Objeto tipo/dinamico, estructura, que guarda; algoritmo; invariante/correctitud; complejidad
preprocess/query/update/mem. Palabras magicas: bloque contiguo, propiedad de ciclo, flujo en arbol, greedy earliest subsequence, rollback snapshot.

MANBER-MYERS
len=1;2..n, sufijo i+1 > par(rank[i],rank[i]+len]). Ordenar pares; pares iguales mismo newRank. Correcto porque compara primera mitad y si empata segunda mitad. O(nlog*2n) con sort; O(nlogn) optimizado.

DISTINCT SUBSTRINGS
Total substrings $n(n+1)/2$. Sufijo SA[i] aporta n-SA[i] prefijos; LCP[i] ya aparecieron con anterior.
Ans=sum(n-SA[i]-LCP[i]).

REFRAIN
Array ints. Maximizar len*freq de una subcadena. SA sobre array; LCP histogram. Un intervalo de sufijos con min LCP>=L da freq=width+1. Largest rectangle over LCP da max L*freq; recuperar posicion de sufijo.

IMPERIAL ROADS
MST T peso W. Parzar e=(u,v,w): T+e crea ciclo. Quitar max edge del path. T(u,v).
ans=W+w-maxEdge(u,v). Correctitud: propiedad de ciclo; para incluir e la unica decision es que arista quitar del ciclo.

ARPA PARITY TREE
Si labels en camino y pregunta puede formar palindromo: usar mask xor de paridades root->v. Path mask = pref[u] xor pref[v] xor bit((label[ca]) segun vertices. Palindromo iff popcount(mask)<=1.

RMQ PSEUDO
build lg, st[0]=A, for k: for i: st[k][i]=min(st[k-1][i],st[k-1][i+2*(k-1)]). Query: k=lg[len]. Si piden posicion del minimo, guardar par(valor,pos).

BANANA SA EX
Banana sufijos orden: a(5), ana(3), anana(1), banana(0), na(4), nana(2). SA=[5,3,1,0,4,2]. rank(1)=2 porque anana es tercer sufijo.

LOWER/UPPER P
Para prefijo P, lower busca primer suffix >= P. Upper puede buscar P con ultimo char aumentado o usar comparador has_prefix. En teoria basta intervalo contiguo.

KMPIZ CUANDO
Si una sola cadena y bordes/prefijos: KMP prefix pi o Z. Si muchas comparaciones arbitrary substrings: SA+LCP+RMQ. Si muchos patrones: Aho-Corasick.

SPARSE TABLE
Static, sin updates, op idempotente. st[k][i]=op(A[i..i+2*(k-1)]); transicion dos mitades. [l,r]; len=r-i+1,k=lg[len], op(st[k][l],st[k][r-2*(k+1)]). [l,r]: len=r-l. O(nlogn)/O(1).

HLD CORE
heavy child=max subtree. Light edge reduce size al menos mitad -> O(logn) light por root-node. Camino u-v se parte en O(logn) segmentos de cadenas. pos/head/parent/depth/out. Con segtree path O(log*2n).

KASAI
Para i texto: p=rank[i], vecino prev j=SA[p-1]. Si antes LCP era h, al mover i+1 baja <=1. while S[i+h]==S[j+h] h++ . LCP[p]=h; h=max(h-1,0). O(n).

LCS TWO STRINGS
SA de A+Bs. Max LCP vecino de origen distinto. Si empaté lexicografico, escoger candidato con menor orden SA. Separadores no deben aparecer en strings.

PERIODIC SUBSTRING
Periodo p valido en substring [l,l+p) si lcp(l,l+p)>=len-p. Max k=len/p. Problema avanzado usa runs; SA+LCP/LCS verifica igualdad O(1). En examen explica runs/periodos, no brute force.

TREE PATH SUBSEQ
HLD ordena U->V en segmentos. Para cada char c, lista posiciones en base. Mantener puntero en S. En segmento forward usar lower_bound; reverse usar upper_bound anterior. Greedy primera aparicion es optimo para subsecuencia.

SUBSEQ PATH PSEUDO
segmentos=decompose. path ordered(U,V). idx=0. for seg in segments: while idx<|S| and find S[idx] in seg after current: consume. YES iff idx==|S|.

HLD ORDER PATH
U->LCA se guarda en segmentos upward. LCA->V se guarda y luego se invierte para respetar orden. Cuidado con incluir LCA una sola vez si se concatenan letras.

LCA PATH MAX
Si edge weights en MST: mx[v][0]=w(parent,v). Al subir 2^i: mx[v][i]=max(mx[v][i-1],mx[up[v][i-1]][i-1]).

IMPERIAL EDGE IN MST
Si e ya esta en MST, maxEdge(u,v)=w si consideras path sin e? En query de arista MST, respuesta debe W. Formula W+w-max path entre u,v en MST incluye esa misma arista como maxima al menos w; funciona si path es el path MST.

RMQ PROOF
Induccion en k: bloque 2^k = dos bloques 2^(k-1). Query: dos bloques elegidos cubren todo rango; solape permitido por idempotencia op(x,x)=x. Si op=sum NO sirve con solape; usar segment tree/Fenwick.

HLD PITFALLS
Si valores en ansas, guardar peso(parent[v],v) en pos[v]; al consultar tramo final excluir pos[ca]. Subtree(u) es intervalo [in[u],out[u]] si DFS/HLD asigna subarbol contiguo.

LCP FORMULA
Si rank[i]=a< b=rank[j], lcp(i,j)=min(LCP[a+1..b]). Prueba: en SA, LCP comun de extremos queda limitado por minimo entre vecinos; todos comparten prefijo de longitud ese minimo. Preprocesa RMQ.

SUBSTRING COMPARE
A=(1..r),B=(2..z). x=min(lcp(1,1z),lenA,lenB). Si x<minLen compara char siguiente. Si x==minLen, mas corta menor. Si iguales y statement pide, desempata por (l,r).

FREQUENCY STRING
Para patron m y k occ sorted. Min substring con k ocurrencias = min |j occ[i+k-1] occ[j]+|m|. Si occ<k -> -1. Muchos patrones: Aho-Corasick para recolectar occ; ocurrencias pueden solaparse.

TRIE LEX ORDER
DFS preorder con hijos ordenados por caracter. Visitar nodo antes que hijos porque string prefix < extension. Hijo char menor antes que mayor. Lowercase: iterar 26 letras ==> O(n).

LCP PAIR SUM STACK
Para suña minimos subarrays: stack pares(h,count), cur=sum minimos subarrays terminando aqui. Al meter h, pop >=h restando height*count, acum count, push(h,count), cur+=h*count,total+=cur.

LCS CORRECT
Cualquier common substring es prefijo de un sufijo de A y uno de B. En el intervalo de sufijos que comparten ese prefijo hay vecinos de origen distinto, por eso mirar vecinos basta.

COMPARATOR PROOF
Orden lex depende del primer caracter distinto. LCP da cantidad igual inicial. Si no hay distinto antes de que una termine, la corta es prefijo y menor.

DISTINCT PROOF
Al recorrer SA, todo prefijo de sufijo actual es substring. Los prefijos de longitud <=LCP con anterior ya fueron vistos; los mayores son nuevos.

LCA BINARY
up[v][i]=ancestro 2^i: mx[v][i]=max edge al subir. Igualar depth; subir ambos si up difiere; ultimo paso incluye edge a LCA. dist(u,v)=dep[u]+dep[v]-2dep[ca]. O(logn).

SA/RANK
SA[p]=indice del p-esimo sufijo en orden. rank[i]=p tal que SA[p]=i. Rank es orden/clase, no caracter. Sufijos con mismo prefijo P quedan consecutivos en SA.

PATTERN SEARCH
P en texto T: L=primer sufijo >=P. R=primer sufijo que ya no tiene prefijo P. Yes iff L<R. Count=R-L. Occ positions=SA[L..R-1]. Simple O(|P|logn).

BORDERS ALL SUBSTR
Borde no vacio de longitud b en substring equivale a dos copias iguales length b en extremos -> par de sufijos con lcp>=b. Total = n(n+1)/2 + sum_{i<j}lcp(i,j). Sum lcp pares = sum minima de subarrays LCP con pila monotona.

FOOLS ROADS
Cada query path u-v: cnt[u]++,cnt[v]++,cnt[ca]=2. Postorder acumula. sub[x] cuenta rutas que cruzan edge parent[x]-x. O((n+q)logn).

TREE AND QUERIES
DSU on tree/shall-to-large: para cada subarbol responder colores/frecuencias. Mantener hijo pesado, borrar contribuciones de hijos livianos, conservar pesado, agregar livianos. Cada nodo se re-agrega O(logn) o amort O(nlogn)/O(n).

FOOLS EDGE ANSWER
Si aristas estan numeradas, al DFS desde child x asigna ans[id(parent,x)] = acc[x]. Root no tiene edge. cnt[ca]=2 no -1/-1 porque el camino entra y sale del LCA hacia ambos lados.

DSU ON TREE PROOF
Shall-to-large: cada vez que un elemento se mueve a una bolsa mayor, el tamano al menos se duplica, por tanto se mueve O(logn) veces.

TRIE VARIANT
Si comparacion lex dice palabra corta antes de extension, preorder. Si root vacio no se pide, omitir root pero mantener preorder para hijos.

EXAM PHRASES
"Preproceso porque estructura es estatica". "La query se reduce a RMQ/LCA". "El intervalo en SA es contiguo". "El rollback restaura invariante por rama".

CARA B - DYNAMIC + INTEGER + EXTRA

CENTROID COUNT PATHS
Count pairs dist<=c. En centroid c: collect distancias por child. ans += count(all) - sum count(child) usando sort+two pointers. Recurse en componentes. Cada path contado en primer centroid que separa endpoints. $O(n\log^2 2n)$.

EULER TOUR TREE
Forest dinamica como secuencia Euler en BST balanceado. connected si representantes BST iguales. link=splice tours; cut=split/recombine. Agregados componente/subtree en nodo raiz del BST. Mejor para component/subtree que path.

VEB OPERATIONS
Split x=(high,low) with sqrt(u). Store min,max, summary of nonempty clusters, cluster[h]. succ: if x<min return min; else try succ low in cluster; else next cluster from summary and its min. T(u)=T(sqrtu)+O(1).

SIGNATURE SORT
Para w grande: partir en bloques, reemplazar digitos presentes por hashes/firmas $O(\log n)$, reducir bits recursivamente y aplicar packed/radix. Idea: usar enteros para romper comparaciones.

WHAT TO PRIORITIZE
Si ves string: intenta SA/LCP interval, formula sum LCP, comparator. Si ves tree path: LCA/HLD/difference. Si ves add/delete over time: offline intervals+rollback. Si ves integer: state n,u,w and pick vEB/Yfast/radix.

CENTROID TWO POINTER
sort D, i=0, r=len(D)-1, while i<r: if D[i]+D[r]<=C: ans+=r-i, i++ else r--. Para contar caminos desde centroid a nodo, incluir distancia 0 del centroid.

CENTROID PROOF
Al quitar centroid, cualquier path con endpoints en componentes distintas pasa por centroid y se cuenta en all. Los endpoints en misma componente se restan y se cuentan en recursion.

ROLLBACK INTERVALS
Al leer ops: add edge guarda start. remove cierra [start,t-1]. Al final, edges abiertas cierran [start,q]. Query en hoja t.

OFFLINE REVERSA
Solo deletions + connectivity: empieza con grafo final, procesa operaciones al revés; delete se vuelve add. DSU responde conectividad.

ROLLBACK DSU
Offline add/delete/query. Edge -> intervalos vida. Intervalo a nodos de segtree tiempo. DFS: snapshot, union del nodo, hoja responde, rollback. No path compression. Union by size. Correcto: path root-hoja contiene aristas activas.

LINK-CUT TREE
Preferred paths: en splay, access(v) expone root-v. makeroot(u)+access(v) expone path u-v; agregado esta en raiz del splay. link/cut/path query $O(\log n)$ amort. Bueno para dynamic tree path min/max/sum.

PREDECESSOR PROBLEM
Maintain S: insert/delete/member/pred/succ. BST $O(\log n)$. vEB $O(\log \log u)=O(\log w)$, space $\Theta(u)$. Y-fast $O(\log w)$ expected/amort, space $O(n)$. Fusion $O(\log_w n)$ theoretical.

COUNTING/RADIX
Counting $O(n+U \cdot \text{digit})$. Radix stable by b-bit digits: $O((n+2^b) \cdot \text{ceil}(w/b))$. Choose $b=\Theta(\log n) \rightarrow O(n^2 w/\log n)$. If $w=O(\log n)$, linear $O(n)$. Stability preserves previous digits.

PROOF PHRASES
Sparse: induction+overlap. SA search: contiguous interval. MST forced edge: cycle property. Fools: flow conservation. HLD: light edge halves. Centroid: first separator counts once. Rollback: segment path equals active set.

ETT CUT/LINK
ETT cut de edge separa tour en partes con split; link concatena tours insertando ocurrencias. Mantiene agregado por BST: size,sum,min. connected por puntero a raiz del treap/splay.

LCT ACCESS IDEA
access(v): hace que el camino raiz-v sea preferido. Cada splay guarda orden por profundidad. Despues makeroot(u), access(v), el splay de v contiene exactamente path u-v.

ONLINE GENERAL DC
Mantener spanning forest por niveles. Al borrar tree edge, buscar replacement non-tree edge; si no existe, componente se separa. ETT mantiene cada forest. Mencionar $O(\log^2 n)$ teorico.

WHEN NOT LCT
Si arbol no cambia: HLD mas simple. Si solo connected: ETT/DSU. Si offline: rollback DSU mas facil y defendible.

DSU STACK
union roots a,b. Si same guardar marcador. Si size[a]<size[b] swap. Push (b,a,sizeA,sumA,components). parent[b]=a; size[a]+=size[b]; sum[a]+=sum[b]; comp--. Rollback restaura.

OFFLINE DECISION
Si te dan todas operaciones o no exigen online -> rollback DSU. Si solo inserta edges -> DSU normal. Si solo borra -> procesar queries al revés insertando. Si online general -> estructuras por niveles/Euler tour.

INTEGER MODEL
n keys, universe $u=2^w$, word w bits. Comparison model lower bound sort $n \log n$. Word RAM: bit/arithmetic $O(1)$. Transdichotomous $w \geq \log n$. AC0 RAM no multiplicacion; cuidado con fusion tricks.

FUSION TREE
Node degree $k=w^{1/5}$. Critical bits distinguish keys. sketch extracts them; packed sketches compare in parallel. Node pred $O(1)$, height $O(\log_k n)=O(\log_w n)$. Mention model assumptions.

METRIC FUNCTIONS
 $\text{sqrt}(x-y)$ metrica porque $\text{sqrt}(a+b) \leq \text{sqrt}(a) + \text{sqrt}(b)$. $\min(1,d)$ metrica si d metrica por truncamiento subaditivo. $|x-y|+|f(x)-f(y)|$ metrica si incluye $|x-y|$ para identidad.

COMMON TRAPS
Half-open vs closed range. Edge values in HLD exclude lca pos. LCP array indexing often $\text{LCP}[i]=\text{between SA}[i-1],\text{SA}[i]$. Do not use path compression with rollback. Use long long for counts n^2/LCP sums.

VEB SPACE FIX
VEB directo $\Theta(u)$. Para universo enorme y sparse, decir "version hash/compressed" o Y-fast. Nunca prometer espacio lineal con vEB basico.

VEB RECURRENCE
 $u \rightarrow \text{sqrt}(u)$ por llamada. Sea $u=2^w$, bits $w \rightarrow w/2$. $T(w)=T(w/2)+O(1)=O(\log w)=O(\log \log u)$.

RADIX STABLE
Ordena de digito menos significativo a mas significativo con counting estable. Estabilidad conserva orden de digitos ya procesados.

SORT CHOICE
u pequeno -> counting $O(n+u)$. $w=O(\log n) \rightarrow \text{radix } O(n)$. w grande teorico -> signature. Solo comparaciones -> mergesort/heapsort $O(n \log n)$.

COMPONENT SUM
DSU guarda sum[root]. query(v)=sum[find(v)]. vertex add en rollback: push old sum[root], sum[root]+=delta; rollback restaura. Yosupo dynamic graph vertex add component sum mezcla intervalos edges + eventos valores.

DYNAMIC CLASSIFY
Static tree path -> HLD/LCA. Dynamic forest path -> LCT. Dynamic forest component/subtree -> ETT. General graph offline -> rollback DSU. General graph online -> spanning forests por niveles + ETT (teorico $O(\log^2 2n)$).

Y-FAST
X-fast stores prefixes of binary trie levels in hash tables -> predecessor by binary searching level. Too much space $O(nw)$. Y-fast stores representatives + buckets size $O(w)$; split/merge buckets; $O(n)$ space.

METRIC CHECK
Metrica: nonneg, identity, symmetry, triangle. Suma metricas es metrica. $(x-y)^2$ falla triangle: $d(0,2)=4 > d(0,1)+d(1,2)=2$. L1,L2,Linf metricas. Pesos positivos L1 metricas.

COMPLEXITY CHEATS
SA MM $O(n \log^2 2n)$ or $O(n \log n)$, Kasai $O(n)$, RMQ build $O(n \log n)/\text{query } O(1)$. HLD $O(n)+O(\log^2 2n)$. LCA $O(n \log n)/O(\log n)$. Rollback intervals $O(m \log q)$. Centroid $O(n \log^2 2n)$.

METRIC TRIANGLE TEST
Para reducir, busca $x+y \leq z$ con directo grande y via intermedio chico. Square falla. Para demostrar, usar desigualdad conocida: $|x-z| \leq |x-y|+|y-z|$.

METRIC SUM PROOF
 $d3=d1+d2$. Nonneg y simetria inmediatas. Identity: suma 0 implica ambas $0 \rightarrow x=y$. Triangle: $d1x+d2x \leq d1xy+d1yz+d2xy+d2yz$.

LONG LONG
Cuentas de substrings, pares de nodos, sumas LCP, caminos: hasta $O(n^2)$. Usar 64-bit en explicacion.

FINAL TRIAGE
Si pregunta strings confirmada: escribe SA/rank/LCP aunque no recuerdes codigo. Si pregunta dynamic: intenta offline rollback. Si integer: define modelo y compara vEB/Yfast/radix.